



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7
з дисципліни
“Технології розроблення програмного забезпечення”

Варіант №7
“Редактор зображень”

Виконав
студент групи ІА–22:
Гордун Д.В.

Перевірив
Мягкий М.Ю.

Київ 2024

Тема: Шаблони «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Мета: Ознайомитися з шаблонами проектування «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD». Реалізувати частину функціоналу з допомогою одного з шаблонів.

Хід роботи

Додав код за посиланням:

<https://github.com/DmytroH25/trpz/pull/6/commits/9c28c2d0823fe6b44c7faa5dce7482e5e139b1e3>

Короткі теоретичні відомості

Принципи проектування:

Принцип Don't Repeat Yourself (DRY) свідчить, що в початкових кодах програмного продукту не повинно бути повторень. Ці повторення можуть спостерігатися на різних рівнях - від повторень наборів рядків коду (абсолютно аналогічні рядки в різних місцях проекту) до різної реалізації функціональності, що повторюється. Повторень уникати слід з кількох причин: передусім, код без повторень значно менше і значно легше читається; по-друге, при знаходженні помилки в коді, що має повторення, є високий шанс не знати/забути виправити цю ж помилку у іншому місці; по-третє, при створенні ідентичного коду ви не лише копіюєте функціональність, але і помилки; по-четверте, при необхідності додавання нових можливостей скрізь де є повторення необхідно внести відповідні коректури. Повторення легко виправити застосувавши відповідну техніку рефакторинга - винесення метода/інтерфейса/класа та ін.

Принцип Keep it simple, Stupid! (KISS) пропонує уникати зайвого ускладнення компонентів системи. Вважається, що система, яка складається з безлічі маленьких простих частин, працює значно надійніше, ніж одна велика складна система. Даною філософією слідує співтовариство Unix, де кожна програма виконує рівно одну функцію, але при цьому виконує її абсолютно добре (Do one thing right). Це також сприяє задоволенню відомого афоризму «The system obviously has no errors rather than system has no obvious errors» (очевидно, що в системі немає помилок; в системі немає очевидних помилок). Використання простих конструкцій спрощує читаємість і сприйняття вихідних кодів.

Принцип You only load it once! (YOLO) вказує на необхідність підвантаження ініціалізаційних і конфігураційних змінних один раз при

запуску програми, щоб уникнути проблем зі швидкістю (повторні зчитування даних з вінчестера).

Принцип Парето застосовується до безлічі різних подій і визначає відсоткове співвідношення 80-20. Декілька прикладів застосування принципу Парето:

- 80% навантаження на сервер створює 20% додатка;
- 80% всього вихідного коду програми пишеться за 20% часу;
- 80% помилок програмного забезпечення можна усунути закривши лише 20% багів. Таким чином, необхідно пам'ятати про те, що для вирішення більшості проблем необхідно докласти лише малу частину зусиль. Однак для доведення додатку до стану досконалості може коштувати величезну суму (80%).

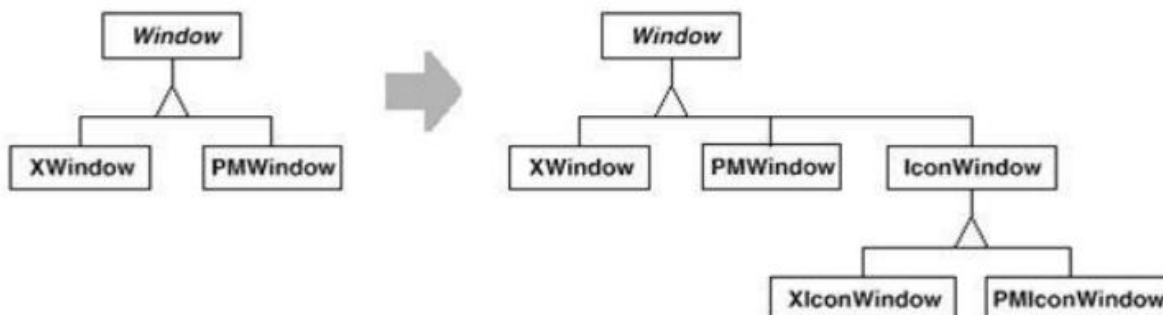
Принцип You ain't gonna need it пропонує відмовитися від деяких рішень в силу того, що вони просто не знадобляться. Досить відомий факт, що програмісти люблять узагальнювати та надавати універсальні рішення на всі випадки життя. Дуже часто досить простого і працюючого рішення замість універсального, оскільки інші випадки навряд чи будуть розглядатися в подальшому. Є необхідний набір функціональності, який необхідно реалізувати; додавання зайвої гнучкості не тільки захарастить код, але і зменшить хід процесу розробки.

Патерн Mediator:

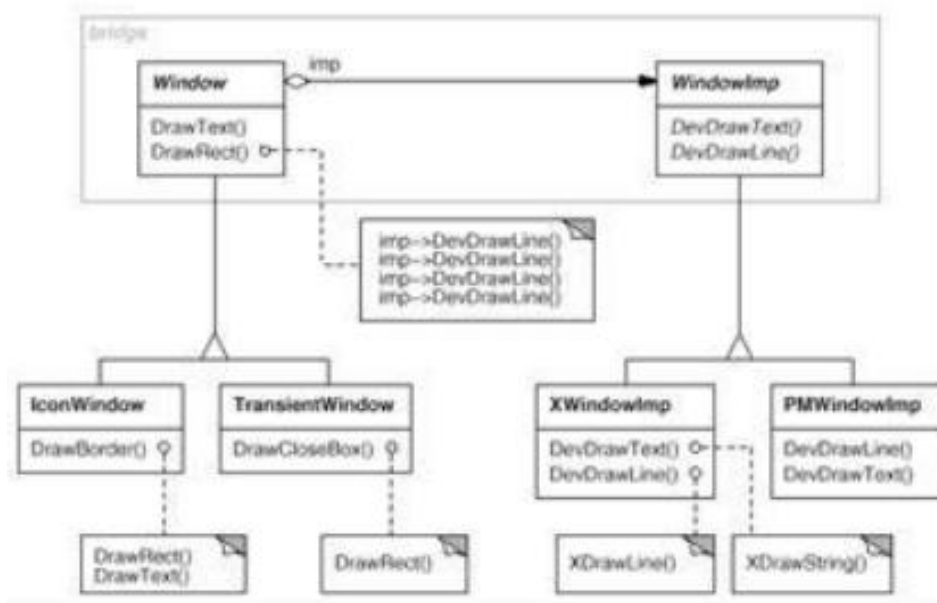
Шаблон «Mediator» (посередник) використовується для визначення взаємодії об'єктів за допомогою іншого об'єкта (замість зберігання посилань один на одного). Даний шаблон схожий на шаблон «команда», проте в даному випадку замість зберігання даних про конкретну дію, зберігаються дані про взаємодії між компонентами. Даний шаблон зручно застосовувати у випадках, коли безліч об'єктів взаємодіє між собою деяким структурованим чином, однак складним для розуміння. У такому випадку вся логіка взаємодії виноситься в окремий об'єкт. Кожен із взаємодіючих об'єктів зберігає посилання на об'єкт «медіатор». «Медіатор» нагадує диригента при управлінні оркестром. Диригент стежить за тим, щоб кожен інструмент грав в правильний час і в злагоді з іншими інструментами. Функції «медіатора» повністю це повторюють.

Патерн Bridge:

Шаблон «Bridge» (міст) використовується для поділу інтерфейсу і його реалізації. Це необхідно у випадках, коли може існувати кілька різних абстракцій, над якими можна проводити дії різними способами. Ось приклад системи без використання шаблону «Міст»:



В даному випадку при появі нових типів вікон буде необхідність створення нових класів їх реалізацій. Це веде до стрімкого збільшення кількості класів і плутанини в вихідному коді. Ось так це виглядає з використанням шаблону «Міст»:



Таким чином можна відокремити абстракцію від її реалізацій, і додавати нові абстракції незалежно від реалізацій.

Патерн Template Method:

Шаблон «Template Method» (шаблонний метод) дозволяє реалізувати покроково алгоритм в абстрактному класі, але залишити специфіку реалізації підкласам. Можна привести в приклад формування веб-сторінки: необхідно додати заголовки, вміст сторінки, файли, що додаються, і нижню частину сторінки. Код для додавання вмісту сторінки може бути абстрактним і

реалізовуватися в різних класах - `AspNetCompiler`, `HtmlCompiler`, `PhpCompiler` і т.п. Додавання всіх інших елементів виконується за допомогою вихідного абстрактного класу з алгоритмом. Даний шаблон дещо нагадує шаблон «фабричний метод», однак область його використання абсолютно інша - для покрокового визначення конкретного алгоритму; більш того, даний шаблон не обов'язково створює нові об'єкти - лише визначає послідовність дій.

Патерн Facade:

Шаблон «Facade» (фасад) передбачає створення єдиного уніфікованого способу доступу до підсистеми без розкриття внутрішніх деталей підсистеми. Оскільки підсистема може складатися з безлічі класів, а кількість її функцій - не більше десяти, то щоб уникнути створення «спагеті-коду» (коли все тісно пов'язано між собою) виділяють один загальний інтерфейс доступу, здатний правильним чином звертатися до внутрішніх деталей. Це також відволікає користувачів від змін в підсистемі (внутрішня реалізація може змінюватися, а наданої послуги немає), що також скоротить кількість змін в використовуваних фасад класах (без фасаду довелося б змінювати вихідні коди в безлічі точок). Звичайно, твердої умови повного закриття внутрішніх класів підсистеми не стоїть - при необхідності можна звертатися до окремих класів безпосередньо, минаючи об'єкт фасад.

Реалізація шаблону Facade в проекті

Шаблон **Facade** використовується для спрощення доступу до складної системи, об'єднуючи різні компоненти в одному інтерфейсі

У нашому випадку Facade виконує роль проміжної ланки між контролерами і сервісами. З використанням цього паттерну ми можемо спостерігати скорочення зв'язків між компонентами.

Контролер працює з одним фасадом, а не безпосередньо з усіма сервісами. Також логіка взаємодії з різними сервісами інкапсульована у фасаді, що робить контролери простішими.

```

@Component
public class ImageFacade {

    private final ImageService imageService;
    private final UserService userService;

    @Autowired
    public ImageFacade(ImageService imageService, UserService userService) {
        this.imageService = imageService;
        this.userService = userService;
    }

    public Image uploadImage(MultipartFile file, Long userId) throws IOException {
        User user = userService.getUserById(userId);
        if (user == null) {
            throw new IllegalArgumentException("User not found");
        }
        return imageService.saveImage(file, user);
    }

    public byte[] rotateImage(Long imageId, double degree) throws IOException {
        imageService.setCurrentState(new RotateState(degree));
        Long editedImageId = imageService.applyEffect(imageId);
        return getImageDataById(editedImageId);
    }

    public byte[] resizeImage(Long imageId, int width, int height) throws IOException {
        imageService.setCurrentState(new ResizeState(width, height));
        Long editedImageId = imageService.applyEffect(imageId);
        return getImageDataById(editedImageId);
    }
}

```

Рис.1 Частина коду, який реалізує шаблон Facade

Висновок: На цій лабораторній роботі було досліджено та кілька шаблонів проектування, таких як «**Mediator**», «**Facade**», «**Memento**», «**Bridge**», «**Template Method**», а також розглянуто такі принципи проектування, як **DRY**, **KISS**, **YOLO**, **Принцип Парето** і **YAGNI**. Кожен із цих підходів зробив внесок у створення гнучкого, масштабованого та підтримуваного програмного забезпечення, але особливий акцент було зроблено на шаблоні **Facade** через його безпосереднє застосування в нашому проєкті. Шаблон **Facade** виконує роль з'єднувальної ланки, яка інкапсулює взаємодію між контролерами та сервісами. Це спрощує архітектуру додатку, робить її більш модульною і полегшує впровадження нових функціональностей.